

Section 2

Week 3 — Transaction Basics

1. Why Transactions Matter

Scenario: A customer rents a DVD. The system must:

Step 1: INSERT a row into the rental table

Step 2: UPDATE the payment table to record the payment

Step 3: UPDATE the inventory availability

Think about it: What if the system crashes after Step 1 but before Step 2?

- The customer has a rental on record but **never paid** — the store loses money.
- Or worse: payment is recorded but no rental — the customer is charged for nothing.
- This is the **fundamental problem** transactions solve.

2. ACID Properties

A — Atomicity ("All or Nothing")

- A transaction is one **indivisible unit** of work.
- Either **all** operations succeed, or **none** of them take effect.

DVD Rental Example:

```
BEGIN;
```

```
-- Customer rents a DVD: 3 steps that MUST all succeed
```

```
INSERT INTO rental (rental_date, inventory_id, customer_id, staff_id)
```

```
VALUES (NOW(), 1525, 130, 1);
```

```

INSERT INTO payment (customer_id, staff_id, rental_id, amount, payment_date)
VALUES (130, 1, currval('rental_rental_id_seq'), 4.99, NOW());

-- If this fails, the rental and payment above are ALSO undone

UPDATE inventory SET last_update = NOW() WHERE inventory_id = 1525;

COMMIT;

```

If **any** step fails → everything is rolled back. No orphan records.

3 Real-World Atomicity Examples:

#	Scenario	What Must Be Atomic
1	Flight booking	Reserve seat + charge credit card + send confirmation. If payment fails, the seat reservation must be undone.
2	University enrollment	Register student + deduct tuition + assign section. If section is full, the tuition deduction must be reversed.
3	E-commerce order	Create order + reduce stock + charge customer. If stock is 0, the charge must not happen.

C — Consistency

- A transaction moves the database from one **valid state** to another **valid state**.
- All constraints (CHECK, FOREIGN KEY, UNIQUE, NOT NULL) must hold after the transaction.
- If a transaction would violate a constraint, the entire transaction is rejected.

DVD Rental Example:

```

-- The rental table has a FOREIGN KEY: customer_id REFERENCES customer(customer_id)

BEGIN;

INSERT INTO rental (rental_date, inventory_id, customer_id, staff_id)

```

```
VALUES (NOW(), 1525, 99999, 1); -- customer 99999 doesn't exist!

-- ERROR: insert or update on table "rental" violates foreign key constraint

-- The DB stays consistent – no rental without a valid customer

ROLLBACK;
```

3 Real-World Consistency Examples:

#	Scenario	Constraint Enforced
1	DVD Rental: duplicate rental	Can't rent the same inventory_id if it's already out (UNIQUE constraint or business logic)
2	Bank: negative balance	CHECK(balance >= 0) prevents overdraft — transfer rejected if insufficient funds
3	Hospital: no doctor, no appointment	FOREIGN KEY prevents creating an appointment for a non-existent doctor

I — Isolation

- Concurrent transactions behave **as if they ran one at a time** (serially).
- Transaction A shouldn't see Transaction B's "half-done" work.
- PostgreSQL uses **MVCC** (Multi-Version Concurrency Control) to achieve this.

What is the Problem Without Isolation?

Imagine two staff members at the DVD store working at the same time:

Staff A (Session 1)	Staff B (Session 2)
_____	_____
BEGIN;	BEGIN;
-- Check: is inventory 1525 available?	
SELECT * FROM rental	
WHERE inventory_id = 1525	

```

AND return_date IS NULL;

-- Result: No active rental → Available!

-- Check same DVD

SELECT * FROM rental

WHERE inventory_id = 1525

AND return_date IS NULL;

-- Result: No active rental →
Available!

INSERT INTO rental (...)

VALUES (... , 1525, ...);

COMMIT;

-- PROBLEM: Same DVD rented to 2 customers!

INSERT INTO rental (...)

VALUES (... , 1525, ...);

COMMIT;

```

Without proper isolation, **both** staff members think the DVD is available.

How PostgreSQL Handles This

PostgreSQL has **4 isolation levels** (covered in detail in Weeks 4-5):

Level	What it Prevents	Use Case
READ UNCOMMITTED	Nothing (PostgreSQL treats as READ COMMITTED)	—
READ COMMITTED (default)	Dirty reads	Normal queries, reports
REPEATABLE READ	Dirty reads + Non-repeatable reads	Financial reports, balance checks
SERIALIZABLE	All anomalies (safest)	Critical operations like

Level	What it Prevents	Use Case
		payments

3 Real-World Isolation Examples:

#	Scenario	Why Isolation Matters
1	DVD Rental: two customers renting last copy	Without isolation, both see 1 copy available and both rent it. With <code>SERIALIZABLE</code> isolation, only one succeeds.
2	Airline: two agents booking last seat	Agent A and Agent B both see seat 14A as empty. Without isolation, both assign it. With proper isolation, the second one gets an error.
3	Bank: reading balance during transfer	While 5000 EGP is being transferred from Account A to B, a report query might see: A already deducted (-5000) but B not yet credited — the total looks wrong. Isolation ensures the report sees either the state <code>BEFORE</code> or <code>AFTER</code> the transfer, never the middle.

Preview: These scenarios will be demonstrated live with **two psql sessions** in **Weeks 4-5**.

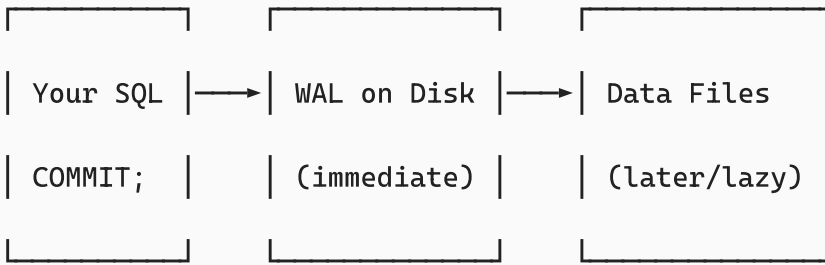
D — Durability

- Once a transaction is **committed**, it **survives crashes** — power failure, server restart, OS crash.
- Even if the server dies **1 millisecond** after `COMMIT`, the data is safe.

How Does PostgreSQL Guarantee This?

PostgreSQL uses **Write-Ahead Logging (WAL)**:

1. You run: `COMMIT;`
2. PostgreSQL writes the changes to the WAL file on disk **FIRST**
3. Only **THEN** does it confirm "`COMMIT`" to you
4. Later, it writes changes to the actual data files (lazily)



▲
|
If crash happens here,

PostgreSQL replays WAL

on restart to recover

Why WAL works: Writing to WAL is **sequential** (fast), while writing to data files is **random** (slow). So PostgreSQL writes to WAL first to guarantee durability, then updates data files in the background.

What Happens on Crash?

Scenario	What Happens
Crash after COMMIT	✅ Data is in WAL → PostgreSQL replays it on restart → Data is safe
Crash before COMMIT	❌ Transaction was never committed → Automatically rolled back → No damage
Crash during COMMIT	✅ If WAL was written → Data is recovered. If not → Rolled back. Either way, the DB is consistent.

DVD Rental Example:

```
BEGIN;  
  
-- Customer returns a DVD  
  
UPDATE rental SET return_date = NOW() WHERE rental_id = 11496;
```

```
-- Staff processes a refund

INSERT INTO payment (customer_id, staff_id, rental_id, amount, payment_date)

VALUES (130, 1, 11496, -2.99, NOW());

COMMIT; -- ← At this exact moment, the data is written to WAL

-- Even if the server loses power RIGHT NOW, this return + refund is SAFE
```

3 Real-World Durability Examples:

#	Scenario	Why Durability Matters
1	DVD Rental: customer returns a film	Customer returns the DVD, staff confirms return in the system, COMMIT succeeds. Power goes out. When the server restarts, the return is still recorded — the customer won't be charged late fees.
2	Online payment: credit card charge	After the payment gateway confirms the charge and the system COMMITs, a server crash cannot "lose" the payment record. The customer won't be double-charged on restart.
3	Hospital: doctor prescribes medication	A doctor enters a prescription, system COMMITs. Server crashes. When it restarts, the prescription is still there — the patient gets the correct medication. Without durability, the prescription is lost and the patient might not be treated.

Preview: WAL and crash recovery are explored hands-on in **Weeks 8-9**.

ACID Summary Table

Property	Question it Answers	Mechanism in PostgreSQL	DVD Rental Example
Atomicity	What if only half the operations succeed?	Transaction rollback	Rental + payment must both happen or neither
Consistency	Does the DB stay valid?	Constraints + triggers	Can't rent to non-existent customer (FK)

Property	Question it Answers	Mechanism in PostgreSQL	DVD Rental Example
Isolation	What if two users act at the same time?	MVCC + isolation levels	Two staff renting the last DVD copy
Durability	What if the server crashes after commit?	WAL (Write-Ahead Log)	Return recorded even after power failure

3. Transaction Syntax in PostgreSQL

Basic Transaction Block (DVD Rental)

```

BEGIN;                                -- Start transaction

-- Customer 130 rents inventory 1525 from staff 1

INSERT INTO rental (rental_date, inventory_id, customer_id, staff_id)

VALUES (NOW(), 1525, 130, 1);

-- Record the payment

INSERT INTO payment (customer_id, staff_id, rental_id, amount, payment_date)

VALUES (130, 1, currval('rental_rental_id_seq'), 4.99, NOW());

COMMIT;                                -- Save all changes permanently

```

Rolling Back

```

BEGIN;

-- Oops! Staff accidentally processes rental for wrong customer

```



```

INSERT INTO rental (rental_date, inventory_id, customer_id, staff_id)

VALUES (NOW(), 1525, 999, 1); -- wrong customer ID!


ROLLBACK;                                -- Undo everything since BEGIN


-- The rental was never recorded

```

After ROLLBACK, it's as if nothing happened. The database is back to its state before BEGIN.

The Three Commands

Command	What it Does
BEGIN	Starts a new transaction block
COMMIT	Saves all changes permanently (end of transaction)
ROLLBACK	Undoes all changes since BEGIN (end of transaction)

4. Autocommit — The Hidden Trap

What is Autocommit?

In PostgreSQL (and in pgAdmin/DBeaver), **every single statement is automatically wrapped in its own transaction** unless you explicitly say BEGIN .

```

-- This runs as its own transaction (auto-committed immediately):

INSERT INTO rental (rental_date, inventory_id, customer_id, staff_id)

VALUES (NOW(), 1525, 130, 1);


-- This also runs as its own separate transaction:

INSERT INTO payment (customer_id, staff_id, rental_id, amount, payment_date)

VALUES (130, 1, currval('rental_rental_id_seq'), 4.99, NOW());

```

Problem: If the connection drops between these two statements, the rental is recorded but the **payment is lost** — the customer gets a free rental!

Why This Matters

Approach	What Happens on Failure
With Autocommit	Rental saved, payment lost → free rental!
With BEGIN/COMMIT	Both undone (ROLLBACK) → no damage

The Fix

Always wrap related operations in an explicit transaction:

```
BEGIN;  
  
INSERT INTO rental (...) VALUES (...);  
  
INSERT INTO payment (...) VALUES (...);  
  
COMMIT;
```

5. Savepoints — Partial Rollback

The Problem with ROLLBACK

ROLLBACK undoes **everything** since BEGIN . But what if you did 10 operations and only the last one was wrong?

Savepoints to the Rescue

A **savepoint** is a bookmark inside a transaction. You can roll back to a savepoint without losing everything.

DVD Rental Example — Processing multiple returns at once:

[illegible]

```

UPDATE rental SET return_date = NOW() WHERE rental_id = 11496;

SAVEPOINT after_first_return;

-- Return DVD 2 (rental_id = 11497)

UPDATE rental SET return_date = NOW() WHERE rental_id = 11497;

SAVEPOINT after_second_return;

-- Return DVD 3 - oops, wrong rental_id!

UPDATE rental SET return_date = NOW() WHERE rental_id = 99999;

-- This rental doesn't exist (0 rows updated) - staff realizes the mistake

ROLLBACK TO after_second_return;

-- Only the third return is undone. Returns 1 and 2 are still pending.

-- Fix: correct rental_id

UPDATE rental SET return_date = NOW() WHERE rental_id = 11498;

COMMIT;

-- All three correct returns are saved.

```

Savepoint Commands

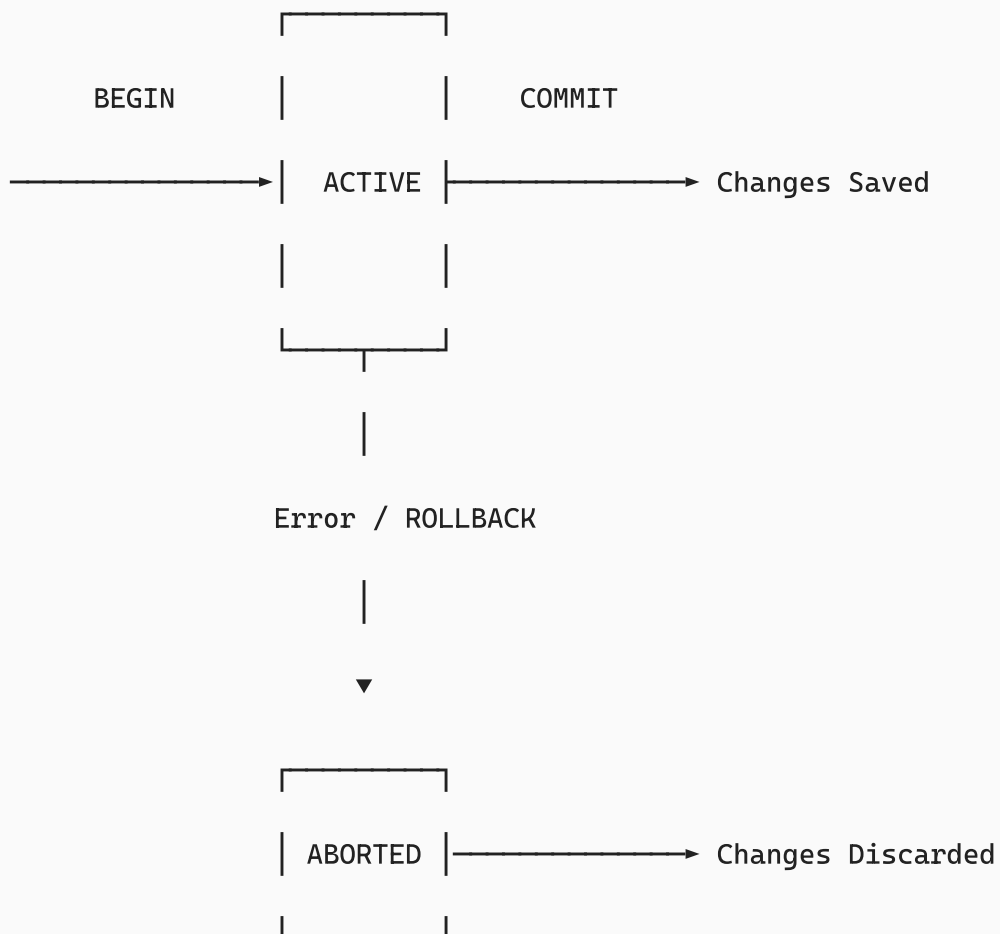
Command	What it Does
SAVEPOINT name	Creates a bookmark with the given name
ROLLBACK TO name	Undoes everything after the savepoint

Command	What it Does
RELEASE SAVEPOINT name	Removes the savepoint (keeps the changes)

When to Use Savepoints

- Error handling in complex multi-step transactions
- "Try this, and if it fails, undo just this part"
- Batch processing: process 100 returns, skip the ones with errors, keep the rest

6. Transaction States — Mental Model



Important: In PostgreSQL, if ANY statement inside a transaction fails (e.g., constraint violation), the entire transaction enters an **ABORTED** state. You **MUST** issue `ROLLBACK` before you can do anything else.

```
BEGIN;

-- Try to insert a rental with a non-existent customer

INSERT INTO rental (rental_date, inventory_id, customer_id, staff_id)

VALUES (NOW(), 1525, 99999, 1);

-- ERROR: violates foreign key constraint


-- Transaction is now ABORTED - ANY query will fail:

SELECT * FROM customer; -- ERROR: current transaction is aborted

ROLLBACK;                -- Must rollback before continuing
```